CRUD HTTP y su implementación mediante Spring-Boot para bd ventas & [Teoría]

Plugins Eclipse necesarios: Spring Tools Suite y Thymeleaf **3**
Alternativa desarrollo thymeleaf con intellij **4**
Creación de un proyecto nuevo con Spring Starter **4**
[Teoría]Tipos de dependencias en Maven **12**
Compile 14
Provided 15
Runtime 15
Test 16
System 17
Import 17
Resumen de ámbitos o alcances de dependencias maven 18
Modelo de base de datos ventas **18**
Diagrama de base de datos 20
Instalación de lombok en el IDE Eclipse **22**
Estrategia MVC **24**
DAO **24**
[Teoría]Componente Spring **31**
[Teoría]Estereotipos de Componente Spring **31**
[Teoría]Inyección de dependencias de Spring **33**
[Teoría]Inversión de Control IoC **34**
[Teoría]Modos de Inyección de Dependencias **35**
[Teoría]Ventajas de la Inyección de Dependencias **37**
Mayor cohesión y menor acoplamiento 37
Reutilización 38
Cambios de arquitectura 38
Pruebas 38
Análisis de DAO con spring-jdbc **39**
Tarea 3.1 - Creación de ComercialDAO con Spring JdbcTemplate y pruebas sobre CommandLineRunner de Spring **45**
[Teoría]Patrón Controlador (@Controller) **46**
Patrón Controlador en ClienteController **46**
[Teoría] Esquema petición Spring MVC **47**
Inyección de bean de Servicio en Controlador **48**
[Teoría]Patrón Servicio (@Service) **49**
Patrón Servicio en ClienteService **50**
[Teoría] Patrón Repositorio (@Repository) **51**

Patrón Repositorio en ClienteDAO y ClienteDAOImpl **52**
Vista mediante plantillas thymeleaf **53**
Thymeleaf **55**
DemoTH **57**
th:text 57
th:block 60
${...} Expresiones de variables 63
*{...} Expresión de selección sobre th:object 64
@{...} Expresión de Enlace (links) 66
th:with variables 67
~{...} y th:fragment | Expresión de fragmento 68
Carga de fragmentos por selectores css 71
Fragmentos parametrizados 72
#{...} Expresiones de mensaje (internacionalización) 76
Formularios 78
th:class, th:classappend (th:style, th:styleappend) y el operador elvis ?: 82
th:if, th:unless 83
th:switch, th:case 85
th:each - loop 86
th:each - state 87
Ejemplos th:each - objeto state 88
MapStruct - Mapper para DTO **90**
Validación con Spring Boot **102**
Interpolación de errores de validación 112
Internacionalización del mensaje de error 113
Validación personalizada (Custom validator) 115
Validación a nivel de clase (Class Level validation) 120
Manejo global de excepciones con @ControllerAdvice **128**
Manejo local de excepciones en @Controller mediante @ExceptionHandler **133**
Test con WebClient **134**

Plugins y extensiones necesarias para Spring Booot y Thymeleaf

- **`Database Client`** o **`MySQL`**: Permite conectarte directamente a tu base de datos MySQL desde la barra lateral de VS Code, ejecutar consultas SQL y ver las tablas sin salir del editor.
- **Extension Pack for Java:** Incluye el soporte básico del lenguaje Java, depurador (*Debugger*), ejecutor de pruebas (JUnit), explorador de proyectos y gestor para Maven/Gradle.
- **Spring Boot Extension Pack:**
    - **Spring Initializr Java Support:** Permite generar proyectos de Spring Boot directamente desde la paleta de comandos de VS Code.
    - **Spring Boot Tools:** Proporciona autocompletado avanzado y navegación rápida en archivos de propiedades (`application.properties` / `application.yml`) y clases anotadas.
    - **Spring Boot Dashboard:** Panel lateral para visualizar, iniciar, detener y depurar aplicaciones Spring Boot en ejecución.
- **Lombok Annotations Support:** Necesario si utilizas Lombok para autogenerar getters, setters y constructores.
- **`Thymeleaf`**: Proporciona coloreado de sintaxis, resaltado y autocompletado para atributos `th:*` en los archivos HTML dentro de `src/main/resources/templates`.

Alternativa desarrollo thymeleaf con intellij

https://www.jetbrains.com/help/idea/thymeleaf.html

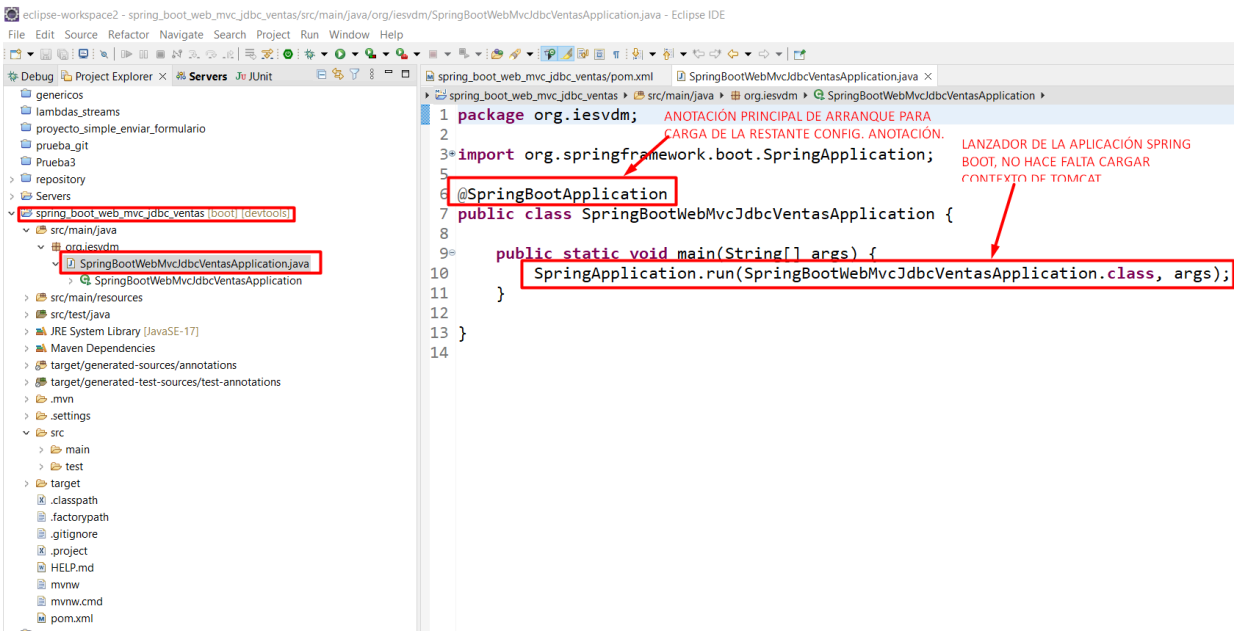
Creación de un proyecto Spring llamado "spring-web-mvc-jdbc-ventas" con Initializr en el entorno elegido (En VSCode con **`Spring Initializr: Create a Maven Project`** indicado en el documento publicado con anterioridad sobre "Creación de proyecto en Spring Initializr"):

- Versión: 4.1.2
- Lenguaje: Java
- Grupo: org.iesvdm
- Artefacto: ventas
- Package: org.iesvdm.ventas
- Package type: Jar
- Java version: 21 (la que tengas instalada)
- Añadir las dependencias:

- Lombok
- Spring Web
- Thymeleaf
- Spring Boot DevTools
- Spring Data JPA
- MySql Driver

Si revisamos la estructura del proyecto:

- Destaca paquete principal **org.iesvdm**.
  - Dentro del paquete principal tenemos la clase de carga del contexto de Spring **VentasApplication**
    - Dado que estamos utilizando la aceleración de Spring Boot, tenemos que esta clase está marcada con la anotación para arranque de toda la configuración de contexto de una Spring Boot Application: **@SpringBootApplication**



Se trata de un proyecto **Maven** por lo que las dependencias se controlan en el fichero **pom.xml**. Automáticamente el starter de Spring Boot nos configura este fichero de dependencias en base a los módulos indicados, en este proyecto:

- Spring Boot DevTools
- Lombok
- JDBC API
- MySQL Driver
- Thymeleaf
- Spring Web

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>4.1.2</version>
       <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>org.iesvdm</groupId>
   <artifactId>ventas</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>ventas</name>
   <description/>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
   <scm>
       <connection/>
       <developerConnection/>
       <tag/>
       <url/>
   </scm>
   <properties>
       <java.version>21</java.version>
```

```
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
<!-- dependencia para implementar vista con motor de plantillas html thymeleaf y su soporte en spring web mvc -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
<!-- dependencia para implementar web-mvc con spring -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
    <!-- añadimos devtools para obtener la funcionalidad de LiveReload para el proyecto -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
<!-- añadimos lombok para mayor simplificación del código -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-thymeleaf-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

[Teoría]Tipos de dependencias en Maven

Hay dos tipos de dependencias en Maven: **directas** y **transitivas**.

Las dependencias **directas** son **las que incluimos explícitamente en el proyecto.**

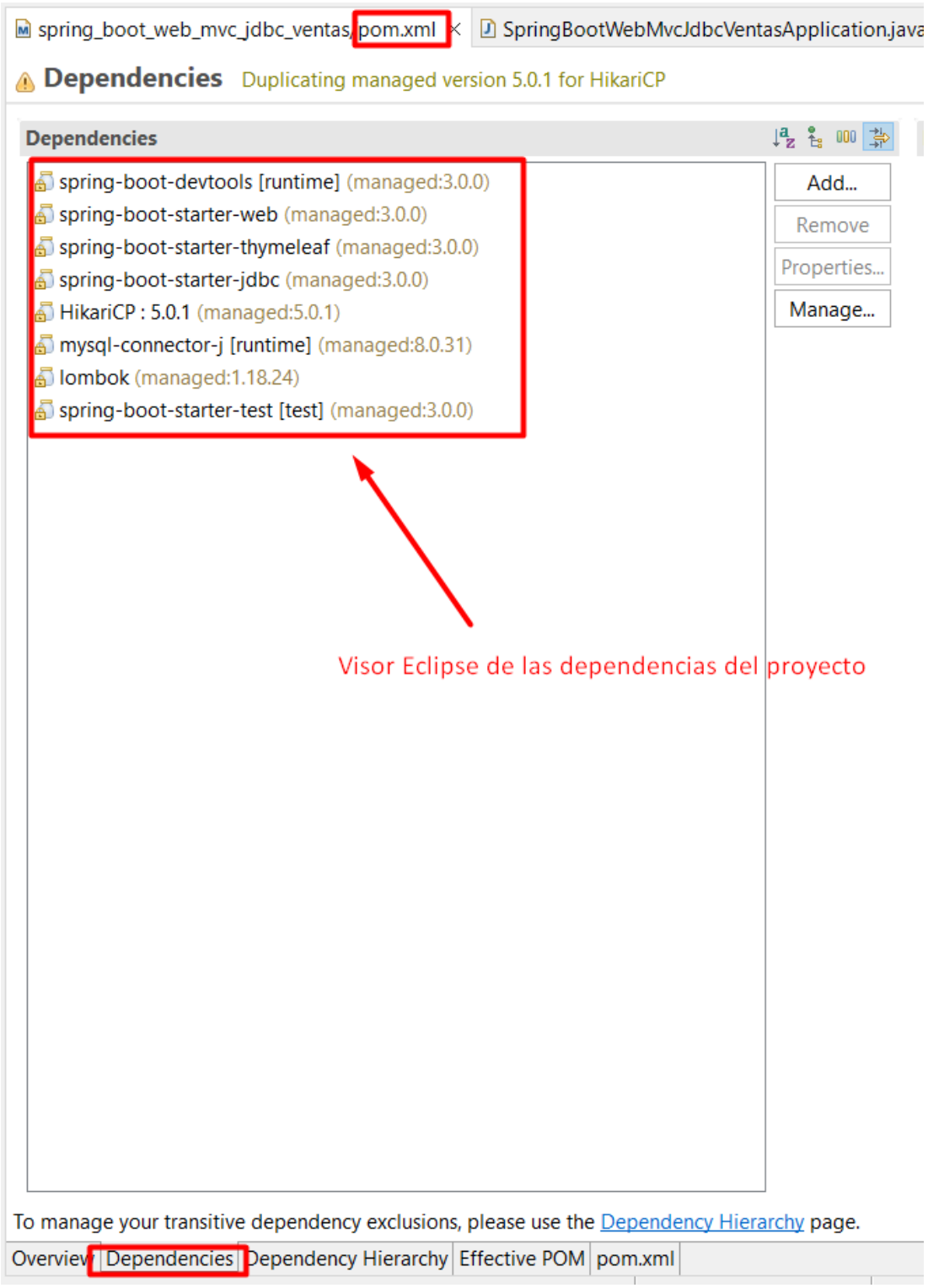
Estos se pueden incluir usando etiquetas <dependency> :

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
</dependency>
```

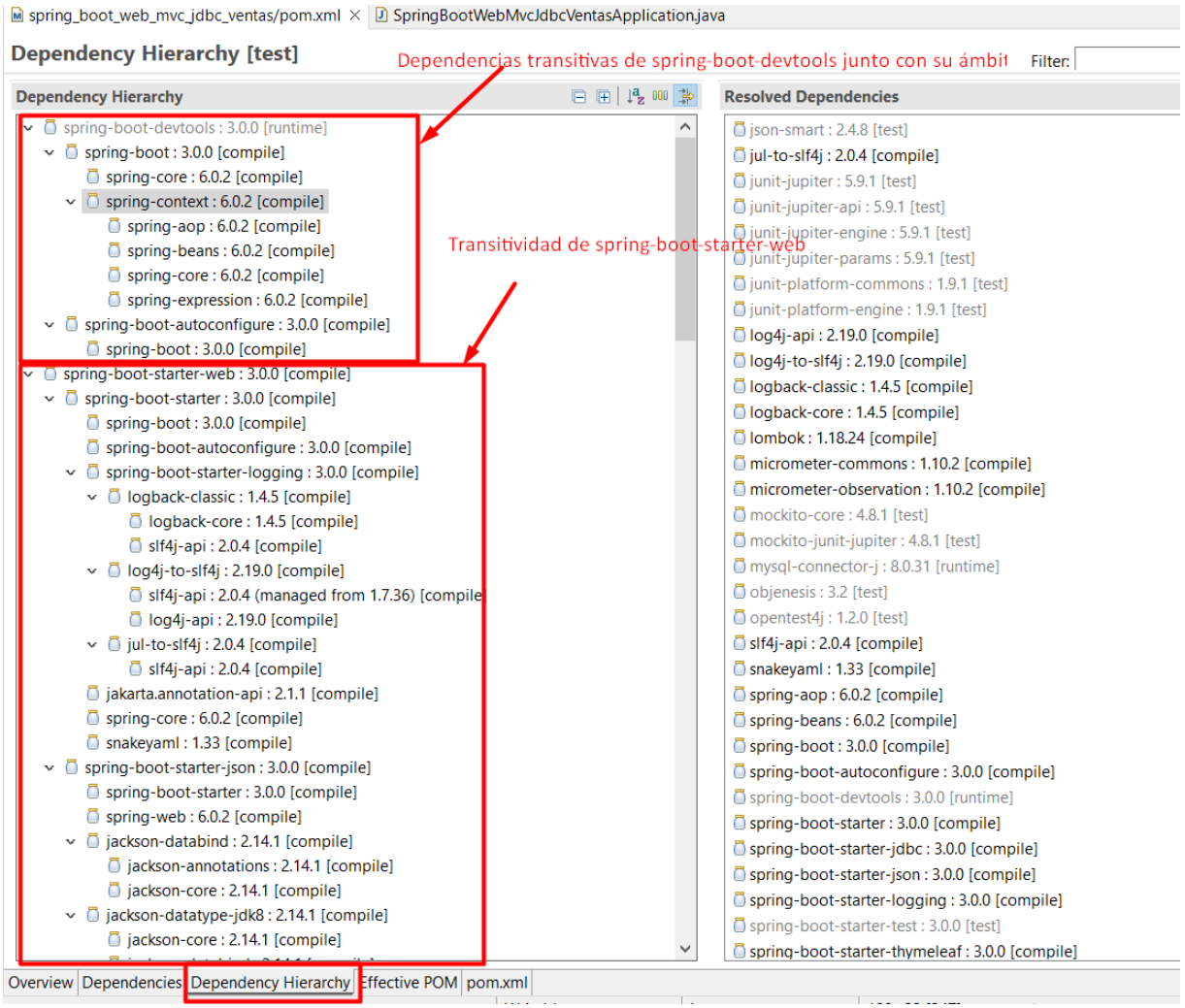
Las dependencias **transitivas** son **requeridas por las dependencias directas**.

Maven incluye automáticamente las dependencias transitivas requeridas en nuestro proyecto.

Podemos enumerar todas las dependencias, incluidas las dependencias transitivas en el proyecto usando el comando **mvn dependency:tree** o en **eclipse** mediante la selección el fichero pom.xml:



Mientras que en la pestaña **Dependency Hierarchy** puedes ver la expansión en dependencias transitivas que necesita el proyecto:



Los **ámbitos de dependencia** pueden ayudar a **limitar** la **transitividad de las dependencias**. También modifican el classpath para diferentes tareas de compilación. Maven tiene seis ámbitos de dependencia predeterminados.

Cada ámbito, excepto *import*, tiene un impacto en las dependencias transitivas.

Compile

Este es el ámbito predeterminado cuando no se proporciona ningún otro ámbito.
Las dependencias con este alcance están disponibles en el classpath del proyecto en todas las tareas de compilación. También se propagan a los proyectos dependientes.
Más importante aún, estas dependencias también son transitivas. Ejemplo:

```
<dependency>
   <groupId>commons-lang</groupId>
   <artifactId>commons-lang</artifactId>
   <version>2.6</version>
</dependency>
```

Provided

Usamos este alcance para marcar **las dependencias que JDK o un contenedor deben proporcionar en tiempo de ejecución.**

Un buen caso de uso para este ámbito sería una aplicación web implementada en algún contenedor, donde el contenedor ya proporciona algunas bibliotecas.

Por ejemplo, podría ser un servidor web que ya proporcione la API de Servlet en tiempo de ejecución.

En nuestro proyecto, podemos definir esas dependencias con el alcance *provided:*

```
<dependency>
   <groupId>javax.servlet</groupId>
   <artifactId>javax.servlet-api</artifactId>
   <version>4.0.1</version>
```

```
    <scope>provided</scope>
</dependency>
```

Las dependencias *provided (proporcionadas)* solo están disponibles en tiempo de compilación y en el *classpath* de prueba del proyecto.

Estas **dependencias no son transitivas**.

Runtime

Las dependencias con este ámbito son necesarias en tiempo de ejecución. Pero no los necesitamos para la compilación del código del proyecto. Por eso, las dependencias marcadas con el alcance o ámbito *runtime* estarán presentes en el tiempo de ejecución y en el classpath de prueba en el IDE, pero no estarán en el *classpath* de compilación.

Un controlador JDBC es un buen ejemplo de dependencias que deberían usar el alcance del tiempo de ejecución o *runtime*:

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.28</version>
    <scope>runtime</scope>
</dependency>
```

Test

Usamos este alcance para indicar que la dependencia no es necesaria en el tiempo de ejecución estándar de la aplicación, sino que se usa sólo con fines de prueba.

Las dependencias *de prueba no son transitivas y solo están presentes para classpaths de prueba y ejecución.*

El caso de uso estándar para este alcance es agregar una biblioteca de prueba como JUnit a nuestra aplicación:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

System

***El ámbito del sistema* es muy similar al ámbito *provided*.** La principal diferencia es que el *system* requiere que apuntemos directamente a un jar específico en el sistema.

Este **alcance *del sistema* está obsoleto (deprecated)** .

Lo importante a recordar es que construir el proyecto con dependencias de alcance de system puede fallar en diferentes máquinas si las dependencias no están presentes o están ubicadas en un lugar diferente al que señala *systemPath* :

```
<dependency>
  <groupId>org.iesvdm</groupId>
  <artifactId>custom-dependency</artifactId>
  <version>1.3.2</version>
  <scope>system</scope>

<systemPath>${project.basedir}/libs/custom-dependency-1.3.2.jar</systemPath>
</dependency>
```

Import

Solo está disponible para el tipo de dependencia *pom* .

import indica que esta dependencia debe reemplazarse con todas las dependencias efectivas declaradas en su POM.

Aquí, a continuación, la dependencia *del proyecto personalizado (custom-project)* se reemplazará con todas las dependencias declaradas en la sección *pom.xml <dependencyManagement>* del proyecto personalizado .

```
<dependency>
   <groupId>org.iesvdm</groupId>
   <artifactId>custom-project</artifactId>
   <version>1.3.2</version>
   <type>pom</type>
   <scope>import</scope>
</dependency>
```

Resumen de ámbitos o alcances de dependencias maven

Cada ámbito de dependencia afecta a las dependencias transitivas a su manera. Esto significa que diferentes dependencias transitivas pueden terminar en el proyecto con diferentes alcances.

Las dependencias con alcances *provided* y *test* nunca se incluirán en el proyecto principal.

Echemos un vistazo detallado:

- Para el alcance ***compile***, todas las dependencias transitivas con alcance de ***runtime*** se incorporarán con el alcance de ***runtime*** en el proyecto, y todas las dependencias con el ámbito de ***compile*** se extraerán con el alcance de ***compile*** en el proyecto.

- Para el ámbito ***provided***, *las dependencias transitivas del ámbito de* ***runtime*** y de *compile* se incorporarán con el ámbito ***provided*** en el proyecto.
- Para el alcance de ***test***, las dependencias transitivas del alcance de ***compile*** y ***runtime*** *se incorporarán con el alcance de* ***test*** en el proyecto.
- Para el ámbito de **runtime**, *las dependencias transitivas del ámbito de* ***runtime*** y de ***compile*** se extraerán con el ámbito de **runtime** en el proyecto.

Modelo de base de datos ventas

Script SQL de creación (lo añadimos en src/main/resources/ventas.mysql y se importa a mysql):

```
DROP DATABASE IF EXISTS ventas;
CREATE DATABASE ventas CHARACTER SET utf8mb4;
USE ventas;

CREATE TABLE cliente (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido1 VARCHAR(100) NOT NULL,
  apellido2 VARCHAR(100),
  ciudad VARCHAR(100),
  categoría INT UNSIGNED
);

CREATE TABLE comercial (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido1 VARCHAR(100) NOT NULL,
  apellido2 VARCHAR(100),
  comisión FLOAT
);

CREATE TABLE pedido (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```

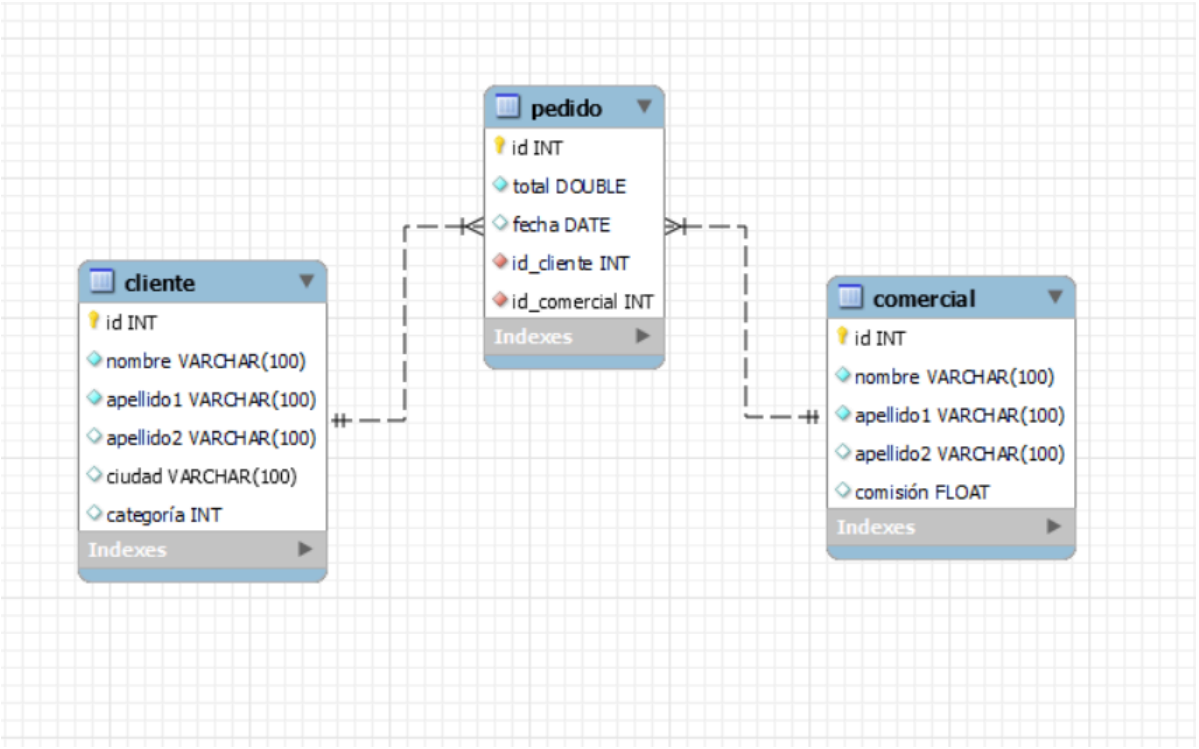
```
  total DOUBLE NOT NULL,
  fecha DATE,
  id_cliente INT UNSIGNED NOT NULL,
  id_comercial INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_cliente) REFERENCES cliente(id),
  FOREIGN KEY (id_comercial) REFERENCES comercial(id)
);

INSERT INTO cliente VALUES(1, 'Aarón', 'Rivero', 'Gómez', 'Almería', 100);
INSERT INTO cliente VALUES(2, 'Adela', 'Salas', 'Díaz', 'Granada', 200);
INSERT INTO cliente VALUES(3, 'Adolfo', 'Rubio', 'Flores', 'Sevilla', NULL);
INSERT INTO cliente VALUES(4, 'Adrián', 'Suárez', NULL, 'Jaén', 300);
INSERT INTO cliente VALUES(5, 'Marcos', 'Loyola', 'Méndez', 'Almería', 200);
INSERT INTO cliente VALUES(6, 'María', 'Santana', 'Moreno', 'Cádiz', 100);
INSERT INTO cliente VALUES(7, 'Pilar', 'Ruiz', NULL, 'Sevilla', 300);
INSERT INTO cliente VALUES(8, 'Pepe', 'Ruiz', 'Santana', 'Huelva', 200);
INSERT INTO cliente VALUES(9, 'Guillermo', 'López', 'Gómez', 'Granada', 225);
INSERT INTO cliente VALUES(10, 'Daniel', 'Santana', 'Loyola', 'Sevilla', 125);

INSERT INTO comercial VALUES(1, 'Daniel', 'Sáez', 'Vega', 0.15);
INSERT INTO comercial VALUES(2, 'Juan', 'Gómez', 'López', 0.13);
INSERT INTO comercial VALUES(3, 'Diego','Flores', 'Salas', 0.11);
INSERT INTO comercial VALUES(4, 'Marta','Herrera', 'Gil', 0.14);
INSERT INTO comercial VALUES(5, 'Antonio','Carretero', 'Ortega', 0.12);
INSERT INTO comercial VALUES(6, 'Manuel','Domínguez', 'Hernández', 0.13);
INSERT INTO comercial VALUES(7, 'Antonio','Vega', 'Hernández', 0.11);
INSERT INTO comercial VALUES(8, 'Alfredo','Ruiz', 'Flores', 0.05);

INSERT INTO pedido VALUES(1, 150.5, '2017-10-05', 5, 2);
INSERT INTO pedido VALUES(2, 270.65, '2016-09-10', 1, 5);
INSERT INTO pedido VALUES(3, 65.26, '2017-10-05', 2, 1);
INSERT INTO pedido VALUES(4, 110.5, '2016-08-17', 8, 3);
INSERT INTO pedido VALUES(5, 948.5, '2017-09-10', 5, 2);
INSERT INTO pedido VALUES(6, 2400.6, '2016-07-27', 7, 1);
INSERT INTO pedido VALUES(7, 5760, '2015-09-10', 2, 1);
INSERT INTO pedido VALUES(8, 1983.43, '2017-10-10', 4, 6);
INSERT INTO pedido VALUES(9, 2480.4, '2016-10-10', 8, 3);
INSERT INTO pedido VALUES(10, 250.45, '2015-06-27', 8, 2);
INSERT INTO pedido VALUES(11, 75.29, '2016-08-17', 3, 7);
INSERT INTO pedido VALUES(12, 3045.6, '2017-04-25', 2, 1);
INSERT INTO pedido VALUES(13, 545.75, '2019-01-25', 6, 1);
INSERT INTO pedido VALUES(14, 145.82, '2017-02-02', 6, 1);
INSERT INTO pedido VALUES(15, 370.85, '2019-03-11', 1, 5);
INSERT INTO pedido VALUES(16, 2389.23, '2019-03-11', 1, 5);
```

Diagrama de base de datos



Beans de modelo

Cada tabla tendrá una clase bean o POJO que la represente en el dominio de programación de Java y constituirán el modelo de datos para Java. Para ello, se crea un paquete org.iesvdm.modelo y dentro se recogen estas clases, por ejemplo, si representamos la entidad de base de datos cliente, tendremos el bean de modelo Cliente con una correspondencia directa con la tabla:

```
package org.iesvdm.modelo;

import lombok.Data;
```

```java
//La anotación @Data de lombok proporcionará el código de:
//getters/setters, toString, equals y hashCode
//propio de los objetos POJOS o tipo Beans
@Data
@SuppressWarnings("unused")
public class Cliente {

	private int id;
	private String nombre;
	private String apellido1;
	private String apellido2;
	private String ciudad;
	private int categoria;

}
```

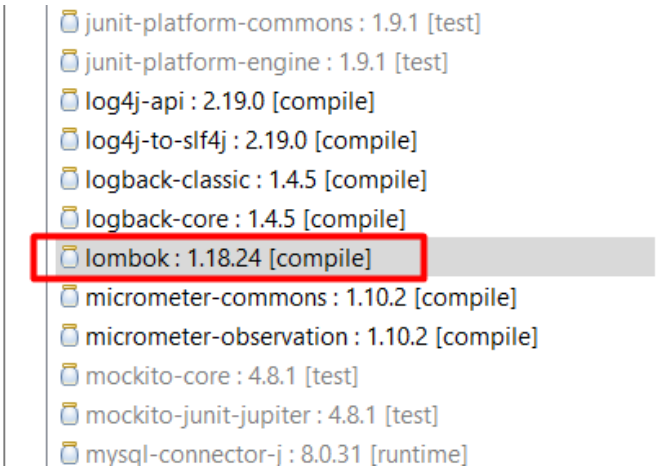
Dado que nuestro proyecto define en su fichero de dependencias pom.xml a la dependencia de lombok podemos hacer un uso fácil de sus anotaciones para simplificar la sobrecarga de código que puede inducir Java. En esta clase de bean de modelo empleamos la anotación @Data https://projectlombok.org/features/Data para asistir en la generación de todo el código complementario al bean.

```xml
<!-- añadimos lombok para mayor simplificación del código -->
<dependency>
	<groupId>org.projectlombok</groupId>
	<artifactId>lombok</artifactId>
	<optional>true</optional>
</dependency>
```

Instalación de lombok en el IDE Eclipse

Para tener operativo lombok en Eclipse necesitamos un paso de instalación, no basta con añadir la dependencia en nuestro pom.xml.

Para ello, descargamos la versión de lombok correspondiente a nuestra dependencia:



De la página del proyecto:
https://projectlombok.org/download

Seguidamente desde consola lanzaríamos el instalador de lombok para el IDE mediante con el comando:
java -jar lombok.jar

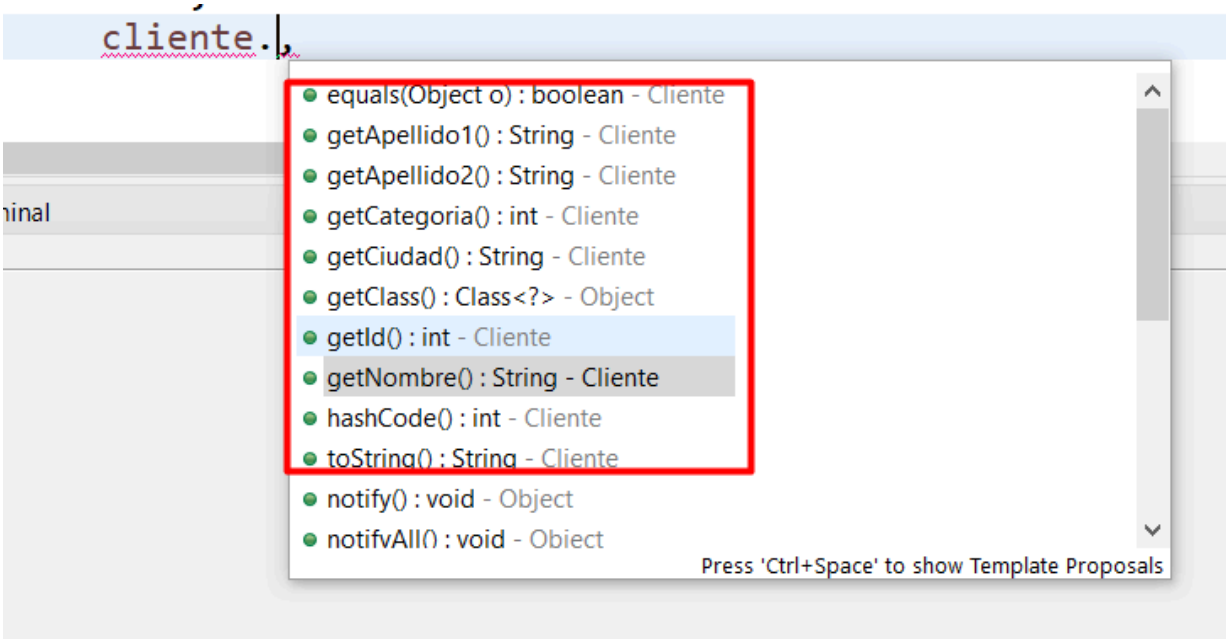


Una posible estrategia de diseño para implementar un CRUD consiste en partir del modelo de base de datos e ir abordando las sucesivas capas del patrón MVC

Después de haber pulsado Install/Update en el IDE correspondiente, se habrá instalado, pidiéndonos que creemos y abramos el IDE de Eclipse:



Tras lo cual ya se tendría la funcionalidad disponible, por ejemplo, si instancio un cliente ya me sugiere todos los métodos:



Estrategia MVC

Una posible estrategia de diseño para implementar un CRUD consiste en partir del modelo de base de datos e ir abordando las sucesivas capas del patrón MVC (Modelo Vista Controlador):

- modelo (beans de datos, DAO),
- controlador (Controlador, Servicio)
- y vista (plantilla html-thymeleaf)

https://docs.spring.io/spring-framework/docs/current/reference/html/web.html#mvc-controller

DAO

En este sentido se hace indispensable implementar el **mínimo de operación CRUD mediante un patrón de interfaz e implementación**. Por un lado estará la interfaz DAO que expone los métodos para hacer CRUD contra la base de datos:

```
package org.iesvdm.dao;

import java.util.List;
import java.util.Optional;
import org.iesvdm.modelo.Cliente;

public interface ClienteDAO {
	public void create(Cliente cliente);
	public List<Cliente> getAll();
	public Optional<Cliente>  find(int id);
	public void update(Cliente cliente);
	public void delete(int id);
}
```

Por otro lado, tenemos la implementación específica de la interfaz CRUD ClienteDAO:

```
package org.iesvdm.dao;

import java.sql.PreparedStatement;
import java.util.List;
import java.util.Optional;

import org.iesvdm.modelo.Cliente;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.jdbc.support.KeyHolder;
import org.springframework.stereotype.Repository;

import lombok.extern.slf4j.Slf4j;

//Anotación lombok para logging (traza) de la aplicación
@Slf4j
//Un Repository es un componente y a su vez un estereotipo de Spring
//que forma parte de la 'capa de persistencia'.
@Repository
public class ClienteDAOImpl implements ClienteDAO {

	//Plantilla jdbc inyectada automáticamente por el framework Spring,
gracias a la anotación @Autowired.
	@Autowired
	private JdbcTemplate jdbcTemplate;

	/**
	 * Inserta en base de datos el nuevo Cliente, actualizando el id en el
bean Cliente.
```

```
     */
    @Override
    public synchronized void create(Cliente cliente) {

                                        //Desde java15+ se tiene la triple
quote """ para bloques de texto como cadenas.
            String sqlInsert = """
                                        INSERT INTO cliente (nombre,
apellido1, apellido2, ciudad, categoría)
                                        VALUES (    ?,      ?,      ?,
?,      ?)
                                    """;

            //Sin recuperación de id generado
//          int rows = jdbcTemplate.update(sqlInsert,
//                                        cliente.getNombre(),
//                                        cliente.getApellido1(),
//                                        cliente.getApellido2(),
//                                        cliente.getCiudad(),
//                                        cliente.getCategoria()
//                                  );

            //Con recuperación de id generado
        KeyHolder keyHolder = new GeneratedKeyHolder();
            int rows = jdbcTemplate.update(connection -> {
                    PreparedStatement ps = connection.prepareStatement(sqlInsert, new String[]
{ "id" });
                    int idx = 1;
                    ps.setString(idx++, cliente.getNombre());
                    ps.setString(idx++, cliente.getApellido1());
                    ps.setString(idx++, cliente.getApellido2());
                    ps.setString(idx++, cliente.getCiudad());
                    ps.setInt(idx, cliente.getCategoria());
                    return ps;
                },keyHolder);
```

```
            cliente.setId(keyHolder.getKey().intValue());

            log.info("Insertados {} registros.", rows);
        }

        /**
         * Devuelve lista con todos los Clientes.
         */
        @Override
        public List<Cliente> getAll() {

            List<Cliente> listFab = jdbcTemplate.query(
                "SELECT * FROM cliente",
                (rs, rowNum) -> new Cliente(rs.getInt("id"),
                                            rs.getString("nombre"),
                                            rs.getString("apellido1"),
                                            rs.getString("apellido2"),
                                            rs.getString("ciudad"),
                                            rs.getInt("categoría")
                                            )
        );

            log.info("Devueltos {} registros.", listFab.size());

        return listFab;

        }

        /**
         * Devuelve Optional de Cliente con el ID dado.
         */
        @Override
        public Optional<Cliente> find(int id) {
```

```
            Cliente fab =  jdbcTemplate
                        .queryForObject("SELECT * FROM cliente WHERE
id = ?"

                                                , (rs, rowNum) -> new
Cliente(rs.getInt("id"),

        rs.getString("nombre"),

        rs.getString("apellido1"),

        rs.getString("apellido2"),

        rs.getString("ciudad"),

        rs.getInt("categoría"))
                                                , id
                                                );

            if (fab != null) {
                    return Optional.of(fab);}
            else {
                    log.info("Cliente no encontrado.");
                    return Optional.empty(); }


    }
    /**
     * Actualiza Cliente con campos del bean Cliente según ID del mismo.
     */
    @Override
    public void update(Cliente cliente) {

            int rows = jdbcTemplate.update("""
                                                UPDATE
cliente SET
```

```
                nombre = ?,
                apellido1 = ?,
                apellido2 = ?,
                ciudad = ?,
                categoría = ?
WHERE id = ?
                                                """,
cliente.getNombre()
                                                ,
cliente.getApellido1()
                                                ,
cliente.getApellido2()
                                                ,
cliente.getCiudad()
                                                ,
cliente.getCategoria()
                                                ,
cliente.getId());
                log.info("Update de Cliente con {} registros actualizados.", rows);

        }

        /**
         * Borra Cliente con ID proporcionado.
         */
        @Override
        public void delete(int id) {
```

```
            int rows = jdbcTemplate.update("DELETE FROM cliente WHERE
id = ?", id);

            log.info("Delete de Cliente con {} registros eliminados.", rows);

        }

}
```

Veamos de qué tipo de componente Spring *esteorotipado* es este DAO:

```
16 //Anotación lombok para logging (traza) de la aplicación
17 @Slf4j
18 //Un Repository es un componente y a su vez un estereotipo de Spring
19 //que forma parte de la 'capa de persistencia'.
20 @Repository  <-- ESTA ANOTACIÓN CONVIERTE A LA CLASE ClienteDAOImpl EN UN COMPONENTE SPRING DISPONIBLE EN SU CONTEXTO
21 public class ClienteDAOImpl implements ClienteDAO {
22
23     //Plantilla jdbc inyectada automáticamente por el framework Spring, gracias a la anotación @Autowired.
24     @Autowired
25     private JdbcTemplate jdbcTemplate;
26
27     /**
28      * Inserta en base de datos el nuevo Cliente, actualizando el id en el bean Cliente.
29      */
30     @Override
31     public synchronized void create(Cliente cliente) {
32
33                             //Desde java15+ se tiene la triple quote """ para bloques de texto como cadenas.
34         String sqlInsert = """
35                             INSERT INTO cliente (nombre, apellido1, apellido2, ciudad, categoría)
36                             VALUES  (     ?,         ?,         ?,       ?,         ?)
37                            """;
```

La anotación de **@Repository** hará que Spring instancie en su contexto de aplicación un objeto (bean de componente de Spring) de modo que otras clases pueden recibir la referencia a este objeto automáticamente ya sea por constructor o mediante mecanismo de anotación @Autowired en el atributo de la referencia).

Más adelante, volveremos sobre el patrón de repositorio previo, pero ahora necesitamos comprender esa naturaleza de componente que maneja el framework de Spring.

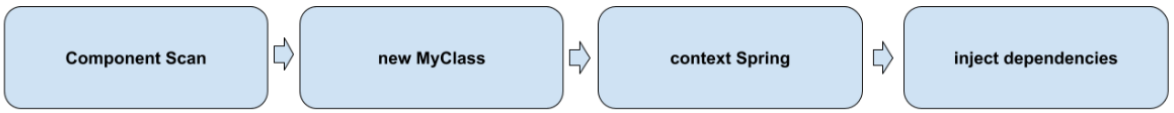
[Teoría]Componente Spring

Un *componente* le permite a Spring detectar *beans* y colocarlos dentro de su contexto para ser utilizados.

Cuando indicamos que una clase es un *@Component*, lo que hacemos es decirle a Spring que queremos que cree una instancia y la gestione.

Al momento de inicializar la aplicación, Spring hará lo siguiente:

- Buscará dentro de la aplicación las clases anotadas con @Component.
- Creará una instancia de los componentes.
- Dejará el componente en su contexto.
- Buscará quién necesita esos componentes y los inyectará en quién los necesite.



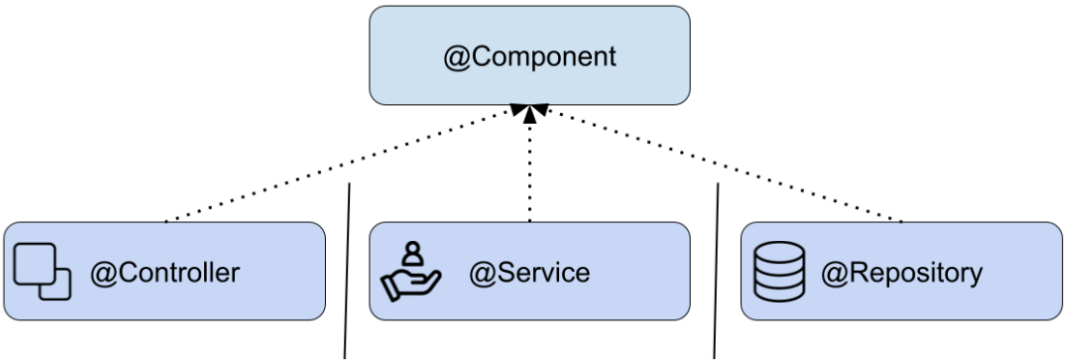
[Teoría]Estereotipos de Componente Spring

Los estereotipos (Stereotypes) son componentes que extienden de @Component.

Los estereotipos más comunes son:

- @Controller / @RestController
- @Service
- @Repository

Todos ellos son también @Component, pero orientados a formar parte en una 'capa' particular.



Un *@Controller* formará parte de la 'capa' de controladores de Spring.

Un *@Service* será parte de la 'capa' de servicios encargado de la funcionalidad del negocio.

Un *@Repository* será parte de la 'capa' de persistencia cuya función será el acceso a los datos.

Observa, por ejemplo, la anotación *@Repository* cómo es un alias de un *@Component*. Lo mismo sucede con el resto de estereotipos.

```
58 @Target(ElementType.TYPE)
59 @Retention(RetentionPolicy.RUNTIME)
60 @Documented
61 @Component
62 public @interface Repository {
63
64     /**
65      * The value may indicate a suggestion for a logical component name,
66      * to be turned into a Spring bean in case of an autodetected component.
67      * @return the suggested component name, if any (or empty String otherwise)
68      */
69     @AliasFor(annotation = Component.class)
70     String value() default "";
71
72 }
73
```

SE TRATA DE UN ALIAS DE @Component

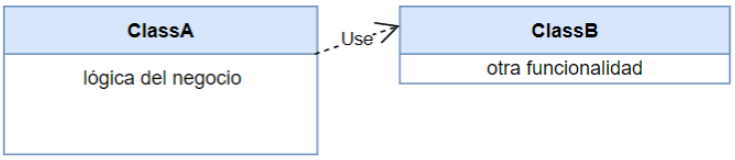
[Teoría]Inyección de dependencias de Spring

La inyección de dependencias es un **patrón de diseño** que tiene como objetivo:

- tomar la responsabilidad de crear las instancias de las clases que otro objeto necesita
- y suministrárselo para que éste los pueda utilizar.

En la inyección de dependencias 'alguien' externo se encarga de las dependencias que las clases necesitan...

Habitualmente las clases dependen de otras para funcionar.



Dependencia

Una clase depende de otra para funcionar.

Por ejemplo, la clase ClassA **depende** de la clase ClassB para lograr el acceso a los datos en la base de datos.

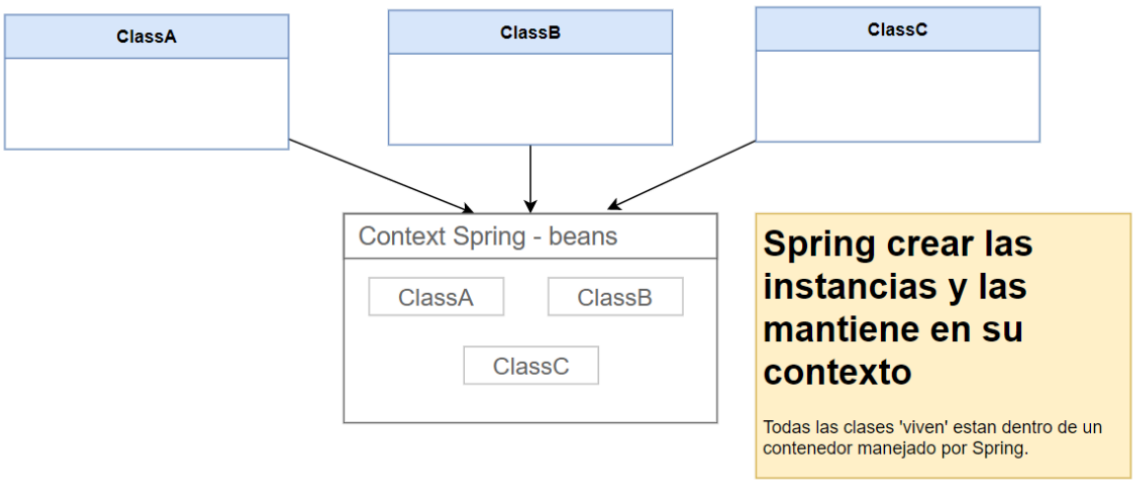
Normalmente, en ClassA crearíamos la ClassB.

ClassA sería la responsable de crear a la ClassB.

[Teoría]Inversión de Control IoC

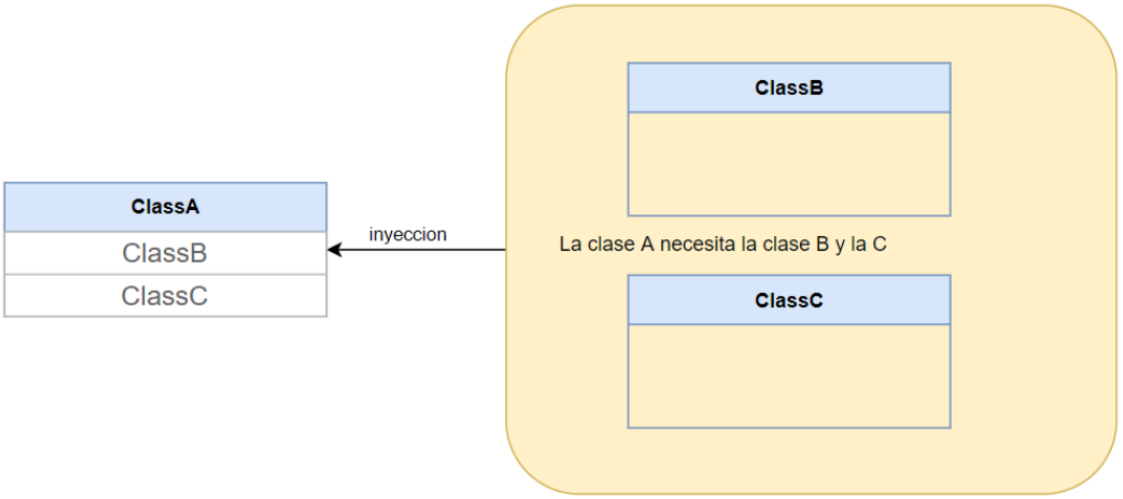
Spring se basa en el concepto o paradigma de IoC (Inversion of Control) mediante la implementación de Inyección de Dependencias (DI) ocupándose de mantener en su 'contexto' todas las instancias de nuestra aplicación y de inyectarle esa instancia a quien la necesite.

En Spring estas instancias se llaman beans. Los beans son las instancias de las clases disponibles para ser utilizados y que se gestionan dentro del contenedor de Spring.



Spring sabe qué dependencias existen entre las instancias y se encarga de satisfacerlas.

Para ello, busca en su contenedor de *beans* la instancia adecuada y se la agrega al objeto cumpliendo así con la inyección de dependencias.



[Teoría]Modos de Inyección de Dependencias

Spring nos da diferentes modos para inyectar las dependencias:

La más común y más recomendada es mediante el uso de *anotaciones*.

La anotación general para indicar que queremos que una clase sea gestionada por el contenedor de Spring es *@Component* .

Al anotar nuestra clase con *@Component* le decimos a Spring que deseamos que esa clase sea un *bean* .

Por lo tanto, será Spring el que hará un *new* de la clase *(instanciar)* y creará la instancia, albergándola en su contexto.

```
@Component
public class ClassB {
    public void doSomething() {
        System.out.println("working");
    }
```

```
}
```

Nuestra clase *ClassA* necesita a la *ClassB*.

Spring se da cuenta de esta dependencia porque nuestra clase es un componente que a su vez tiene en su constructor a la *ClassB* .

Será Spring el que cree la ClassB y luego al instanciar la ClassA le pase en el parámetro del constructor esa dependencia.

```
@Component
public class ClassA {

    private final ClassB classB;

    public ClassA(ClassB classB) {
        this.classB = classB;
    }

    public void doSomething() {
        System.out.println("working");
        classB.doAnotherJob();
    }
}
```

También puede omitirse el paso por constructor utilizando la anotación @Autowired a nivel de atributo o de setter:

```
@Component
public class ClassA {

    @Autowired
    private final ClassB classB;
```

```
    public void doSomething() {
        System.out.println("working");
        classB.doAnotherJob();
    }
}


@Component
public class ClassA {


    private final ClassB classB;

  @Autowired
    public void setClassB(ClassB classB) {
         this.classB = classB
    }

    public void doSomething() {
        System.out.println("working");
        classB.doAnotherJob();
    }
}
```

[Teoría]Ventajas de la Inyección de Dependencias

Mayor cohesión y menor acoplamiento

Siempre que construimos una aplicación debemos pensar en reducir el acoplamiento entre las clases y aumentar la cohesión.

Esto significa que cada clase debe tener una única responsabilidad, debe ser bien específica en dicha responsabilidad y hacerla bien (cohesión). La

reducción del acoplamiento se refiere a que los cambios en una clase no deberían impactar en otra.

Al crear nuestras clases sabiendo que Spring será el encargado de crearlas y de inyectarlas, nos libera para enfocarnos solamente en la funcionalidad de nuestra clase en construcción.

Creamos nuestra clase y definimos sus responsabilidades pensando solo en el ámbito de esa clase. Establecemos cuáles serán los métodos de esa clase para que puedan ser utilizados por otras, pero sin estar pendientes de otras clases.

Reutilización

Una clase está pensada en ser reutilizable por quien la necesite.

Una clase responsable del acceso a la base de datos puede ser utilizada por otras a través del contrato definido en sus métodos y ser inyectada en todas aquellas que necesiten el acceso a la base de datos.

Cambios de arquitectura

Al diseñar las clases pensando en su interfaz. Podemos realizar cambios internos mientras mantengamos el contrato de la clase sin afectar a otras que la están usando.

Pruebas

Las pruebas unitarias sobre las clases son mucho más simples, pudiéndose utilizar *mocks* para aquellas que no son del ámbito de la prueba puntual.

Análisis de DAO con spring-jdbc

Spring tiene un fichero de propiedades **application.properties** donde se configuran los distintos módulos cargados de Spring-Boot.

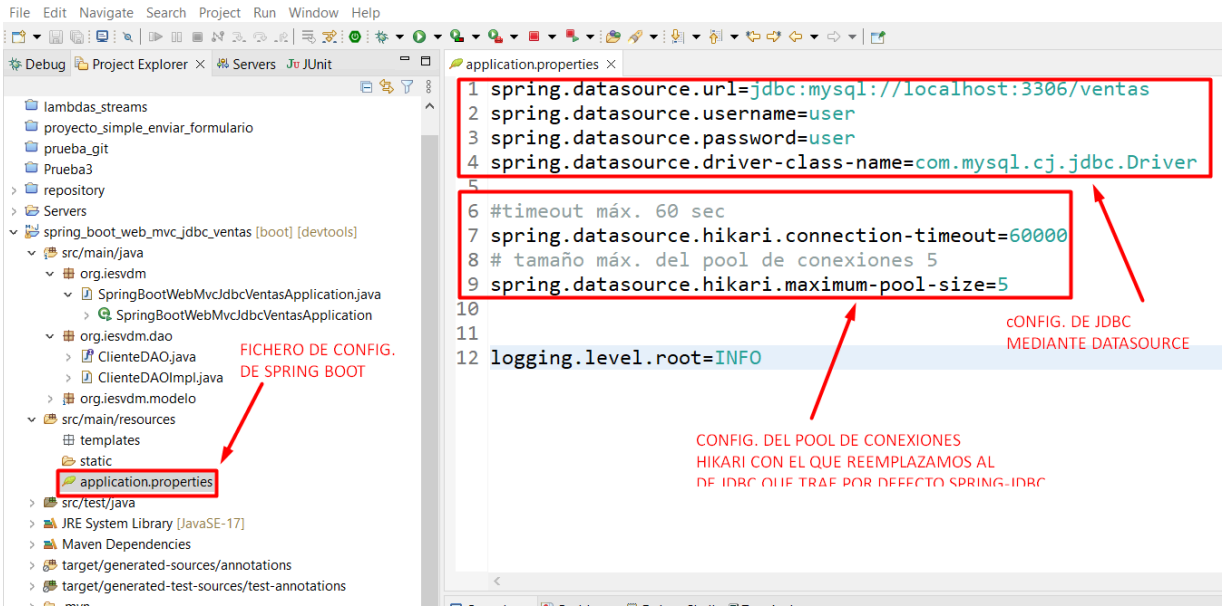
En nuestro caso utilizamos la dependencia spring-boot-starter-jdbc que nos permite una vez configurada en application.properties el datasource para acceso a base de datos emplear la plantilla de spring jdbctemplate.

```
49        <dependency>
50            <groupId>org.springframework.boot</groupId>
51            <artifactId>spring-boot-starter-jdbc</artifactId>
52            <!-- dependencia para utilizar acceso a base de datos
53            con estilo de sql directo sobre jdbc (JdbcTemplate) -->
54            <!-- Excluimos el Connection Pool de jdbc de tomcat,
55            para usar el Hikari Connection Pool más veloz -->
56            <exclusions>
57                <exclusion>
58                    <groupId>org.apache.tomcat</groupId>
59                    <artifactId>tomcat-jdbc</artifactId>
60                </exclusion>
61            </exclusions>
62        </dependency>
63        <!-- añadimos el connection pool Hikari compatible con java11+,
64            automáticamente spring boot lo cogerá en vez
65            del connection pool de tomcat que hemos excluido -->
66        <dependency>
67            <groupId>com.zaxxer</groupId>
68            <artifactId>HikariCP</artifactId>
69            <version>5.0.1</version>
70            <!-- hay que aceptar el warning para compatibilidad con java11+-->
71        </dependency>
72
```

MÓDULO DE SPRING JDBC CARGADO COMO DEPENDENCIA , POR TANTO SE PODRÁ CONFIG. Y UTILIZAR

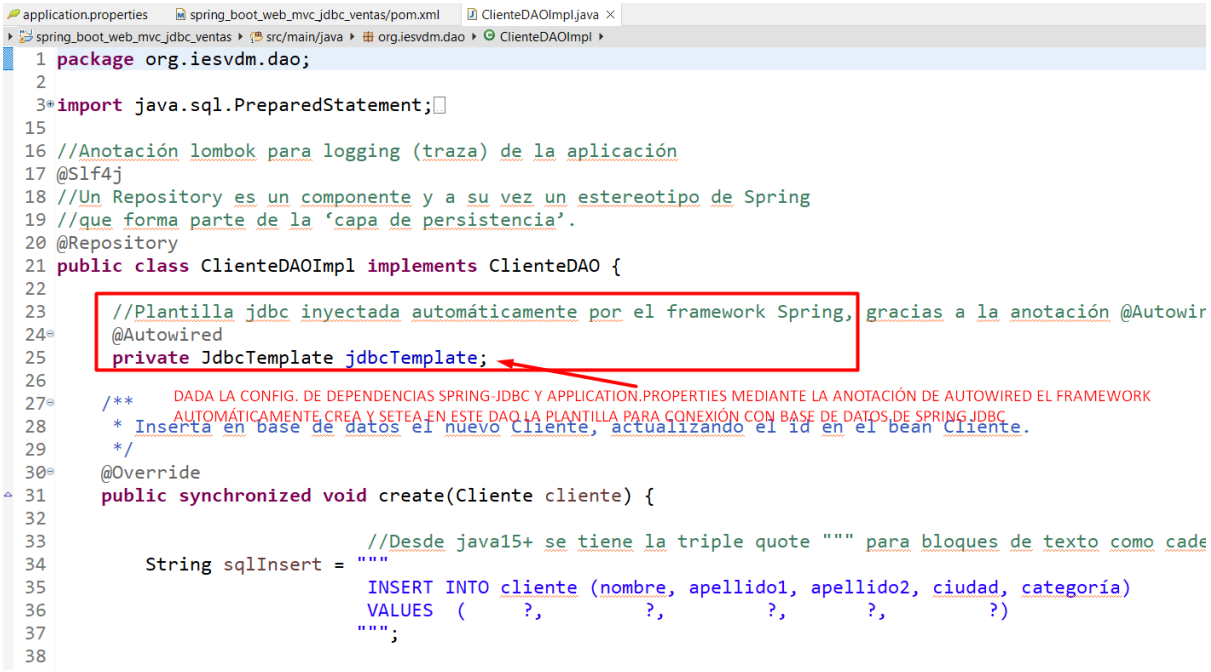
Todo parte de añadir la dependencia apropiada de Spring:

- En este caso, para conectar con base de datos se utiliza spring-boot-starter de jdbc, complementada con la exclusión del Pool de Conexiones de tomcat y la incorporación del Pool de Conexiones Hikari (de mayores prestaciones).



Seguidamente, se añade la configuración externa necesaria para que los módulos de dependencia previos funcionen:

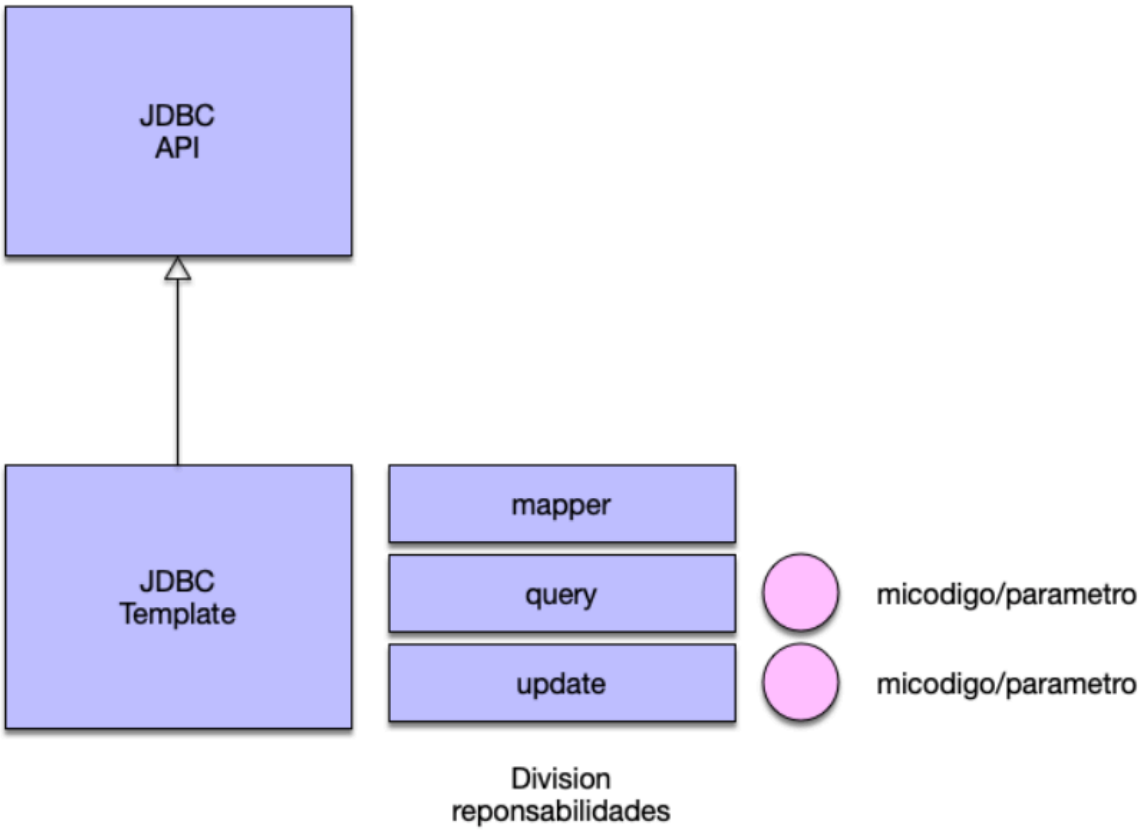
- En este caso, se configura el datasource de jdbc (origen de datos) de la base de datos (parámetros típicos: url, user, passwd y driver-class) y parámetros del Pool de Conexiones Hikari (tamaño máximo del pool y límite de timeout para peticiones de base de datos)



Basta con tener un atributo del tipo JdbcTemplate anotado mediante @Autowired para que el framework inyecte automáticamente la dependencia de jdbcTemplate configurada y lista para uso.

Este recurso de spring: **JdbcTemplate** es un patrón software ***template***, donde la plantilla (template) implementa los detalles específicos control de la conexión. Con esta API tendremos la funcionalidad:

- creación y cierre de conexiones
- ejecución de sentencias y llamadas a procedimientos almacenados
- iterando sobre el ResultSet y devolviendo resultados



Cosas que hace Spring JdbcTemplate (Spring = Primavera :) y cosas que quedarían bajo **Tu** responsabilidad:

Acción	Primavera	Tú
Defina los parámetros de conexión.		X
Abra la conexión.	X	
Especifique la instrucción SQL.		X
Declarar parámetros y proporcionar valores de parámetros		X
Prepare y ejecute la declaración.	X	
Configure el ciclo para iterar a través de los resultados (si los hay).	X	
Haz el trabajo para cada iteración.		X
Procesar cualquier excepción.	X	
Manejar transacciones.	X	
Cierre la conexión, la instrucción y el conjunto de resultados.	X	

JDBC TEMPLATE SIMPLIFICA EL USO
A ESTAS 4 ACCIONES FUNDAMENTALES
POR PARTE DEL PROGRAMADOR,

Spring Framework se ocupa de todos los detalles de bajo nivel que pueden hacer que JDBC sea una API tan tediosa.

Referencia oficial:
https://docs.spring.io/spring-framework/docs/6.0.0/reference/html/data-access.html#jdbc
Versión traducida:
https://docs-spring-io.translate.goog/spring-framework/docs/current/reference/html/data-access.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sc#jdbc

Si revisamos los métodos que nos dan la funcionalidad de CRUD:

- **getAll()**

```java
    /**
     * Devuelve lista con todos loa Clientes.
     */
    @Override
    public List<Cliente> getAll() {

        List<Cliente> listFab = jdbcTemplate.query(
                "SELECT * FROM cliente",
                (rs, rowNum) -> new Cliente(rs.getInt("id"),
                                            rs.getString("nombre"),
                                            rs.getString("apellido1"),
                                            rs.getString("apellido2"),
                                            rs.getString("ciudad"),
                                            rs.getInt("categoría")
                                            )
        );

        log.info("Devueltos {} registros.", listFab.size());

        return listFab;

    }

```

HACE LA MAGIA..

lambda de generación de beans Cliente

Query SQL

- **find(int id)**

```java
    /**
     * Devuelve Optional de Cliente con el ID dado.
     */
    @Override
    public Optional<Cliente> find(int id) {

        Cliente fab =  jdbcTemplate
                .queryForObject("SELECT * FROM cliente WHERE id = ?"
                        , (rs, rowNum) -> new Cliente(rs.getInt("id"),
                                                      rs.getString("nombre
                                                      rs.getString("apelli
                                                      rs.getString("apelli
                                                      rs.getString("ciudad
                                                      rs.getInt("categoría
                        , id
                        );

        if (fab != null) {
            return Optional.of(fab);}
        else {
            log.info("Cliente no encontrado.");
            return Optional.empty(); }

    }
```

Query sql

Lambd
el bea

wrap Optional para el client

1er parámetro (?) asignado a la query,
si hubiera más irían a continuación separados por coma

- update(Cliente cliente)

```java
    /**
     * Actualiza Cliente con campos del bean Cliente según ID del mismo.
     */
    @Override
    public void update(Cliente cliente) {

        int rows = jdbcTemplate.update("""
                                       UPDATE cliente SET
                                                   nombre = ?,
                                                   apellido1 = ?,
                                                   apellido2 = ?,
                                                   ciudad = ?,
                                                   categoría = ?
                                               WHERE id = ?
                                       """, cliente.getNombre()
                                       , cliente.getApellido1()
                                       , cliente.getApellido2()
                                       , cliente.getCiudad()
                                       , cliente.getCategoria()
                                       , cliente.getId());

        log.info("Update de Cliente con {} registros actualizados.", rows);

    }
```

Query Update parametrizada...

Asignación de parámetros (?) por orden de aparición

- **delete(int id)**

```java
    /**
     * Borra Cliente con ID proporcionado.
     */
    @Override
    public void delete(int id) {

        int rows = jdbcTemplate.update("DELETE FROM cliente WHERE id = ?", id);

        log.info("Delete de Cliente con {} registros eliminados.", rows);

    }

```

Query Delete parametrizada...

Asignación de parámetros

- create(Cliente cliente)

```java
    /**
     * Inserta en base de datos el nuevo Cliente, actualizando el id en el bean Cliente.
     */
    @Override
    public synchronized void create(Cliente cliente) {

                            //Desde java15+ se tiene la triple quote """ para bloques de texto como cadenas.
        String sqlInsert = """
                           INSERT INTO cliente (nombre, apellido1, apellido2, ciudad, categoría)
                           VALUES  (     ?,         ?,         ?,       ?,         ?)
                           """;

        KeyHolder keyHolder = new GeneratedKeyHolder();
        //Con recuperación de id generado
        int rows = jdbcTemplate.update(connection -> {
            PreparedStatement ps = connection.prepareStatement(sqlInsert, new String[] { "id" });
            int idx = 1;
            ps.setString(idx++, cliente.getNombre());
            ps.setString(idx++, cliente.getApellido1());
            ps.setString(idx++, cliente.getApellido2());
            ps.setString(idx++, cliente.getCiudad());
            ps.setInt(idx, cliente.getCategoria());
            return ps;
        },keyHolder);

        cliente.setId(keyHolder.getKey().intValue());
```

Insert parametrizado

Recuperador de la clave auto_increment generada por la bd

Seteo de parámetros (?) para el insert por orden

columna con generación de id

Seteo al bean de creación del id generado en base de datos a través del keyHolder

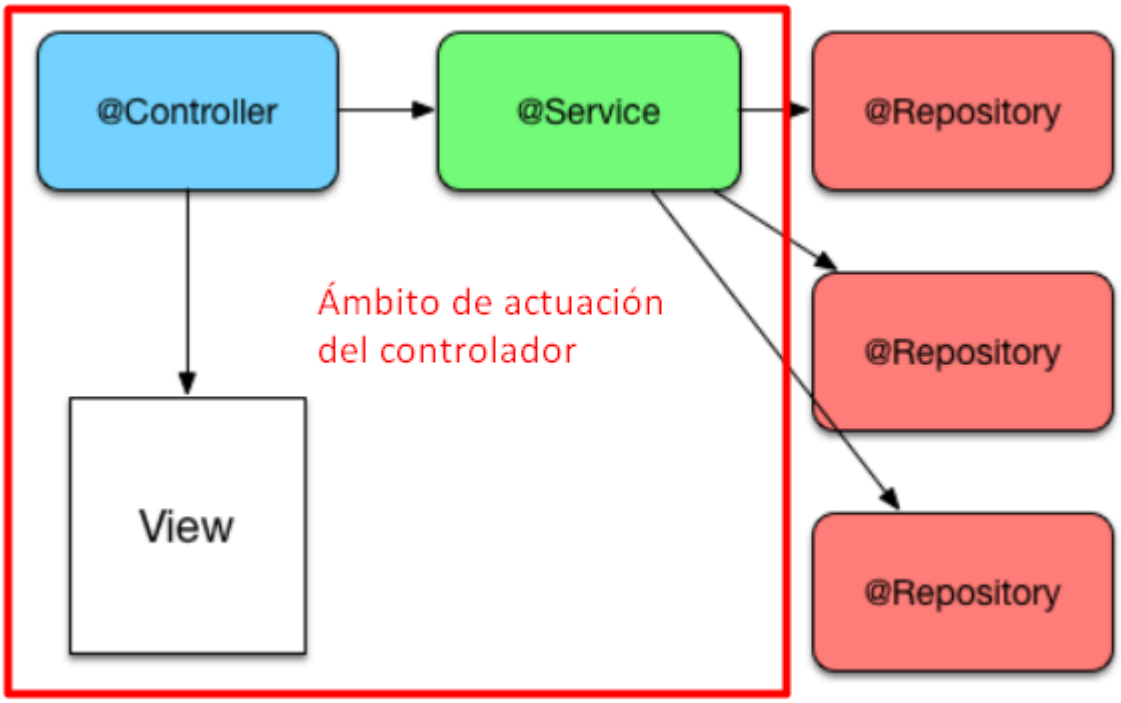
bloque lambda para crear ps con recuperación de id para insert

CommandLineRunner se utiliza para incluir código que se ejecuta al iniciar la aplicación. Se usa el método run para incluir dicho código.

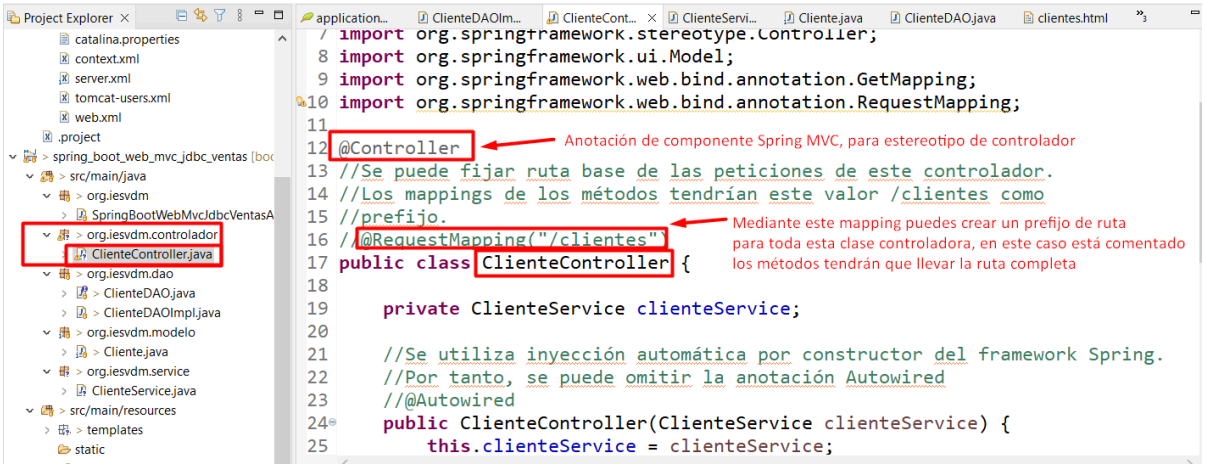


[Teoría]Patrón Controlador (@Controller)

Los controladores se marcan mediante la anotación estereotipada de @Controller. Este estereotipo realiza las tareas de controlador y gestión de la comunicación entre el usuario y la aplicación.



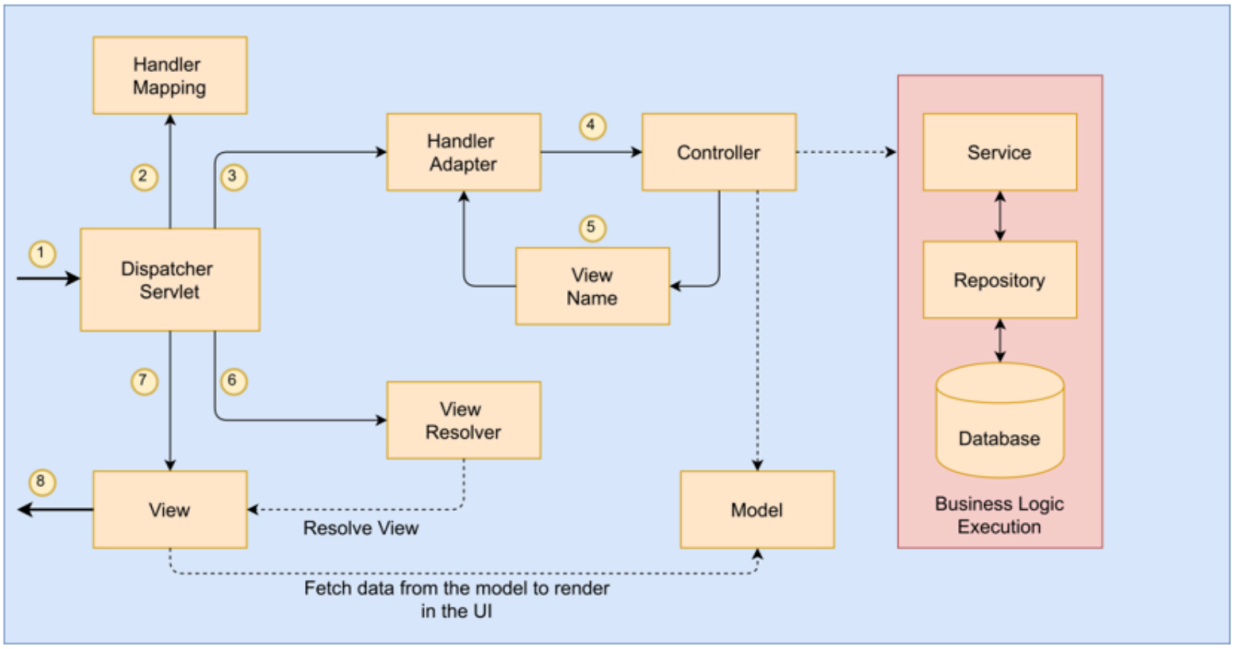
Patrón Controlador en ClienteController



Estos controladores serán los encargados de tratar las peticiones de las diferentes rutas en la aplicación mediante la anotación RequestMapping y sus derivadas: GetMapping, PostMapping,...
https://docs.spring.io/spring-framework/docs/current/reference/html/web.html#mvc-ann-requestmapping

[Teoría] Esquema petición Spring MVC

A continuación, se muestra el diagrama de bloques de una aplicación Spring MVC.En él se observa cómo **Dispatcher Servlet** que implementa Spring MVC en coordinación con el **Handler Mapping** (Manejador de Rutas) redirige las peticiones conforme a la configuración de rutas de cada **Controller** para su tratamiento accediendo al **Model**.



Ciclo de petición a aplicación Spring MVC Web:

1. Una **petición** a Spring MVC se aborda inicialmente por el **servlet despachador** (**Dispatcher Servlet**).
2. Una vez que se recibe una solicitud, el **servlet despachador** primero delega la solicitud entrante a un **HandlerMapping**. El **HandlerMapping** encuentra un **controlador** Spring (**Controller**) configurado para abordar la petición -se trata de un **Controller** con un configuración de ruta que corresponde con la petición).
3. Una vez que se encuentra un controlador, el servlet despachador delega la solicitud a un **HandlerAdapter** que invoca al controlador.

4. Generalmente, un controlador invoca los servicios de lógica de negocio (**Service**) y recupera los datos (**Repository**) de la aplicación.
5. Después de la invocación de la lógica de negocio (capas de servicio y repositorio), un controlador prepara un modelo (**Model**) que contiene los datos de negocio devueltos por las capas de servicio y repositorio.

   Además, el controlador también devuelve un nombre de vista lógica al servlet despachador.
6. El **DispatcherServlet** luego invoca un **ViewResolver** que asigna el nombre de la vista lógica a la vista real (View).
7. La vista devuelta usa los datos del modelo (Model) y los representa, renderizándose la página.
8. Una vez que la vista renderizada (página) enviándose al cliente navegador, la petición se considera como atendida.

Inyección de bean de Servicio en Controlador

```java
@Controller
//Se puede fijar ruta base de las peticiones de este controlador.
//Los mappings de los métodos tendrían este valor /clientes como
//prefijo.
//@RequestMapping("/clientes")
public class ClienteController {

    private ClienteService clienteService;

    //Se utiliza inyección automática por constructor del framework Spring.
    //Por tanto, se puede omitir la anotación Autowired
    //@Autowired
    public ClienteController(ClienteService clienteService) {
        this.clienteService = clienteService;
    }

    //@RequestMapping(value = "/clientes", method = RequestMethod.GET)
    //equivalente a la siguiente anotación
    @GetMapping("/clientes") //Al no tener ruta base para el controlador, cada método tiene que tener la ruta completa
    public String listar(Model model) {

        List<Cliente> listaClientes =  clienteService.listAll();
        model.addAttribute("listaClientes", listaClientes);

        return "clientes";

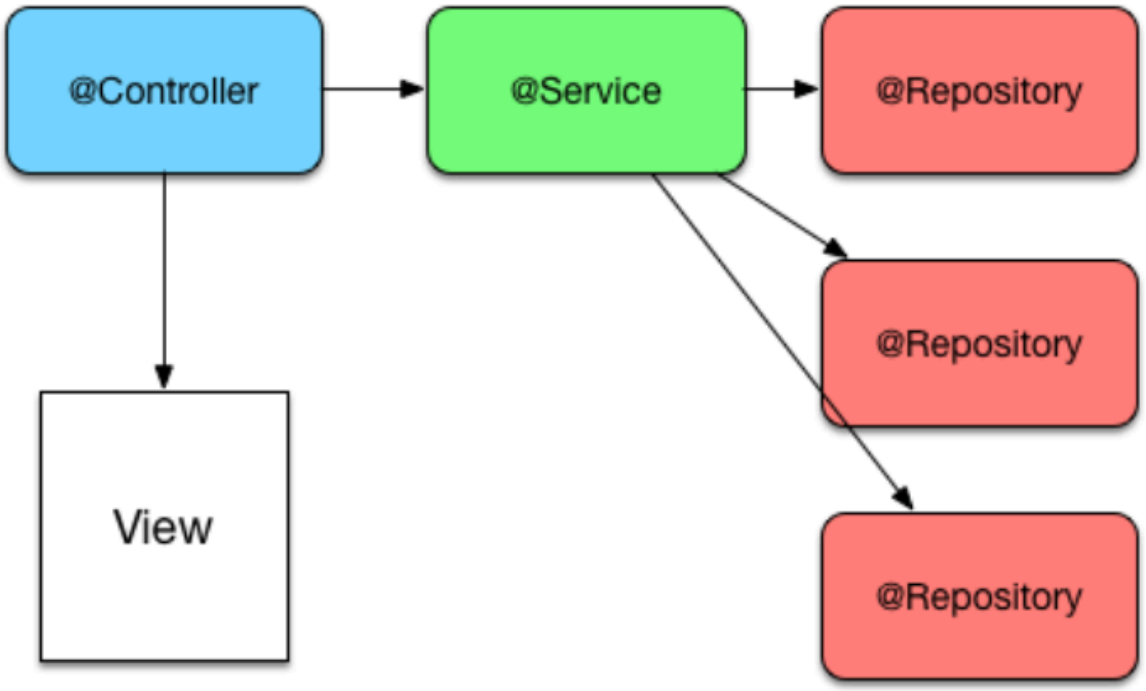
    }

```

Inyección automática de ClienteService por correspondencia de tipo en el constructor del controlador y el uso de la anotación de componente @Service en la clase ClienteService. No hace falta la anotación Autowired, el framework de Spring se encarga la creación del componente y la asignación.

[Teoría]Patrón Servicio (@Service)

@Service es una de las anotaciones más habituales de Spring Framework . Se usa para construir una clase de Servicio que habitualmente se conecta a varios repositorios y agrupa su funcionalidad.



Este estereotipo se encarga de gestionar las operaciones de lógica de negocio más importantes a nivel de la aplicación y **aglutina llamadas a varios repositorios de forma simultánea**. Su tarea fundamental es la de **agregador**.

Patrón Servicio en ClienteService

ClienteService.java × ClienteController.java

```java
package org.iesvdm.service;

import java.util.List;

import org.iesvdm.dao.ClienteDAO;
import org.iesvdm.modelo.Cliente;
import org.springframework.stereotype.Service;

@Service
public class ClienteService {

    private ClienteDAO clienteDAO;

    //Se utiliza inyección automática por constructor del framework Spring.
    //Por tanto, se puede omitir la anotación Autowired
    //@Autowired
    public ClienteService(ClienteDAO clienteDAO) {
        this.clienteDAO = clienteDAO;
    }

    public List<Cliente> listAll() {

        return clienteDAO.getAll();

    }

}
```

Basta con la anotación de componente Service para que el framework de Spring la gestione en su contexto y una instancia la inyecte en ClienteController.

Inyección por constructor de la dependencia de clienteDao. De este modo, no hace falta marcarlo como @Autowired

En el controlador se hace uso de este componente Service: ClienteService mediante la referencia clienteService.

```java
@Controller
//Se puede fijar ruta base de las peticiones de este controlador.
//Los mappings de los métodos tendrían este valor /clientes como
//prefijo.
//@RequestMapping("/clientes")
public class ClienteController {

    private ClienteService clienteService;

    //Se utiliza inyección automática por constructor del framework Spring.
    //Por tanto, se puede omitir la anotación Autowired
    //@Autowired
    public ClienteController(ClienteService clienteService) {
        this.clienteService = clienteService;
    }

    //@RequestMapping(value = "/clientes", method = RequestMethod.GET)
    //equivalente a la siguiente anotación
    @GetMapping("/clientes") //Al no tener ruta base para el controlador, cada método tiene que tener la ruta completa
    public String listar(Model model) {

        List<Cliente> listaClientes =  clienteService.listAll();
        model.addAttribute("listaClientes", listaClientes);

        return "clientes";

    }
```

Esta ruta GET /clientes se mapea a este método listar mediante la anotación @GetMapping. El método hará uso de componente (bean) de servicio clienteService, inyectado por el framework de Spring para obtener la lista de todos los clientes.

A su vez el componente de Service puede delegar a múltiples componentes Repository y agregarlos junto con la lógica de negocio de la aplicación.

```java
@Service
public class ClienteService {

    private ClienteDAO clienteDAO;

    //Se utiliza inyección automática por constructor del framework Spring.
    //Por tanto, se puede omitir la anotación Autowired
    //@Autowired
    public ClienteService(ClienteDAO clienteDAO) {
        this.clienteDAO = clienteDAO;
    }

    public List<Cliente> listAll() {

        return clienteDAO.getAll();

    }


}
```

El service a su vez hace uso del componente DAO o Repository: ClienteDAO.
Hay que visualizar el service como un posible aglutinador de lógica de negocio y accesos a múltples DAO's o componentes de repositorio.

[Teoría] Patrón Repositorio (@Repository)

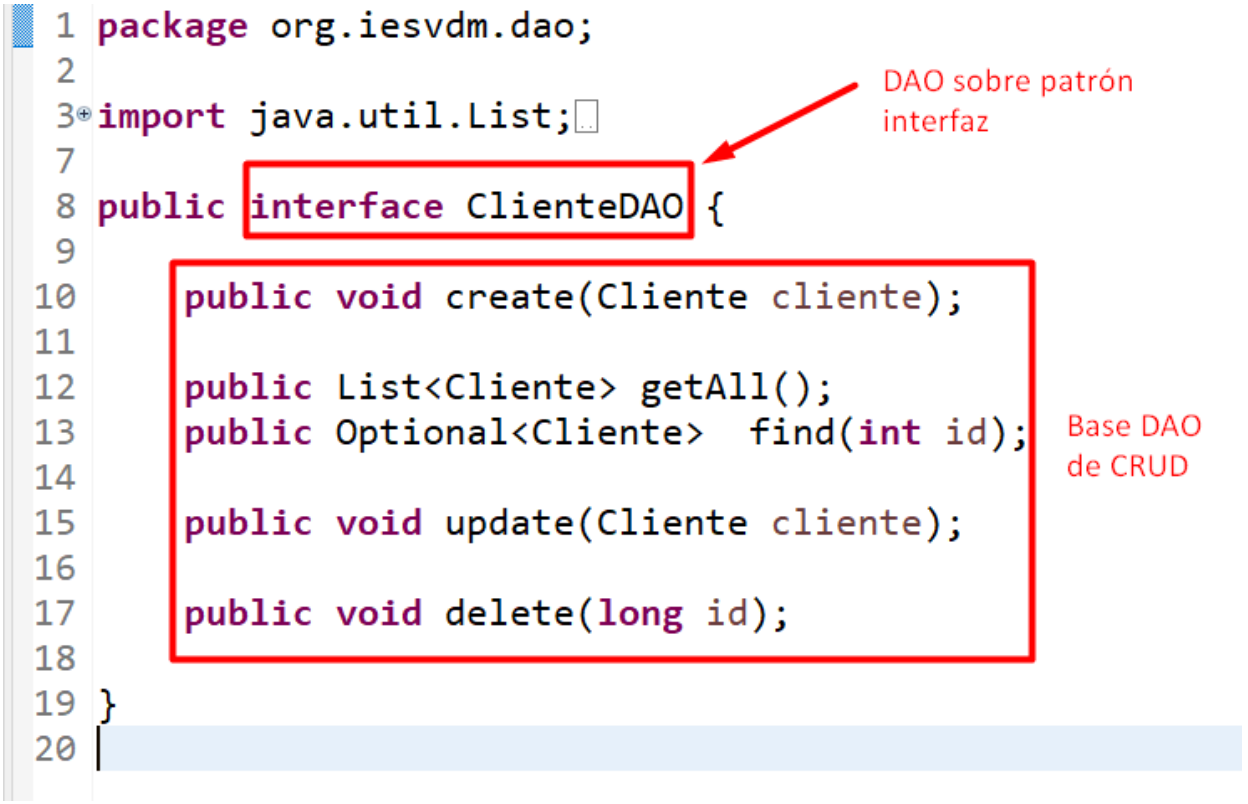
Este estereotipo de componente se encarga de dar de alta un bean para que implemente el patrón repositorio. Este repositorio está encargado de almacenar datos en una base de datos o repositorio de información.

Al marcar el bean con esta anotación, Spring aporta servicios transversales como conversión de tipos de excepciones.



Patrón Repositorio en ClienteDAO y ClienteDAOImpl

El repositorio de cliente seguirá el patrón interfaz con la interfaz java ClienteDAO especificando los métodos básicos de CRUD y su posible ampliación para añadir más lógica o inteligencia de negocio.

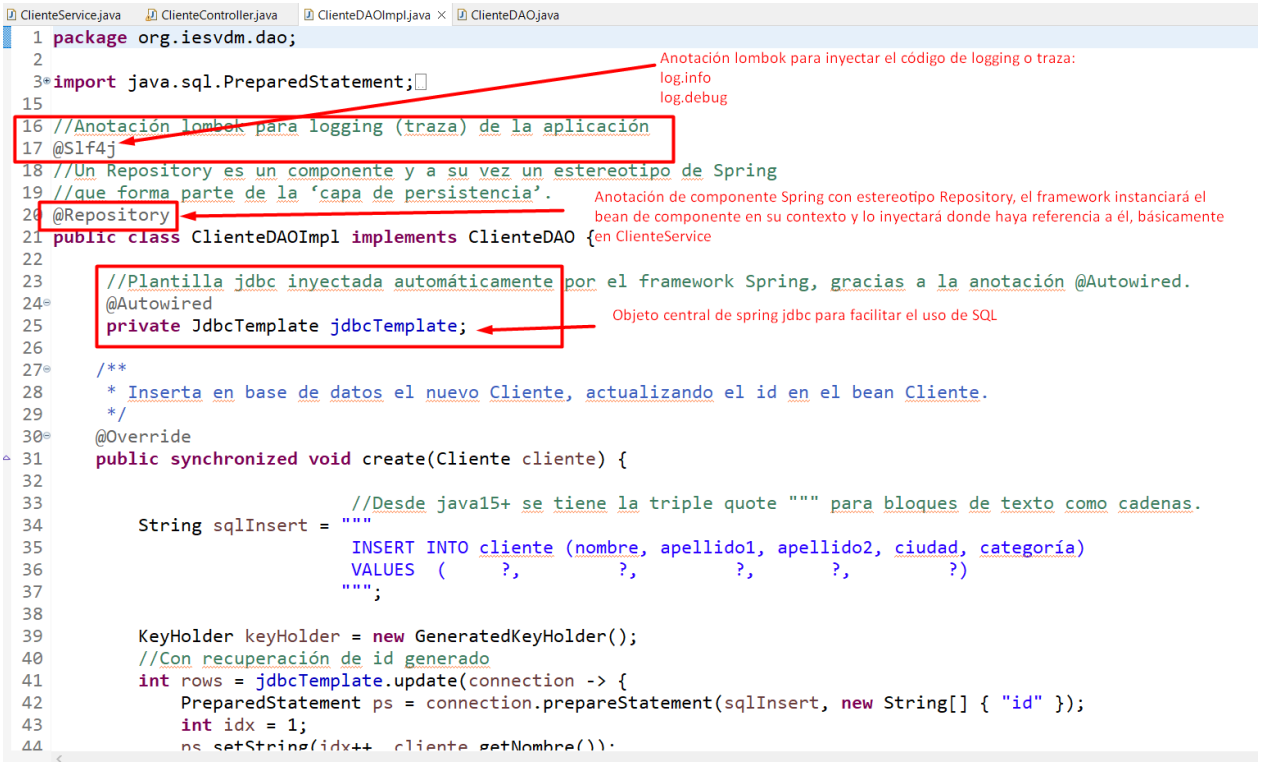



```
1  package org.iesvdm.dao;
2
3  import java.util.List;
7
8  public interface ClienteDAO {
9
10     public void create(Cliente cliente);
11
12     public List<Cliente> getAll();
13     public Optional<Cliente>  find(int id);
14
15     public void update(Cliente cliente);
16
17     public void delete(long id);
18
19 }
20
```

En nuestro caso, dado que hemos cargado el módulo starter de spring boot jdbc la implementación de la interfaz DAO se basará en la **plantilla JDBC de Spring (JdbcTemplate), clase central del módulo para manejo con JDBC.**

Referencias para JdbcTemplate:
https://docs.spring.io/spring-framework/docs/6.0.0/reference/html/data-access.html#jdbc-core
https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/jdbc/core/JdbcTemplate.html

Por tanto, en la implementación de la interfaz **ClienteDAOImpl** tenemos:




```java
package org.iesvdm.dao;

import java.sql.PreparedStatement;

//Anotación lombok para logging (traza) de la aplicación
@Slf4j
//Un Repository es un componente y a su vez un estereotipo de Spring
//que forma parte de la 'capa de persistencia'.
@Repository
public class ClienteDAOImpl implements ClienteDAO {

    //Plantilla jdbc inyectada automáticamente por el framework Spring, gracias a la anotación @Autowired.
    @Autowired
    private JdbcTemplate jdbcTemplate;

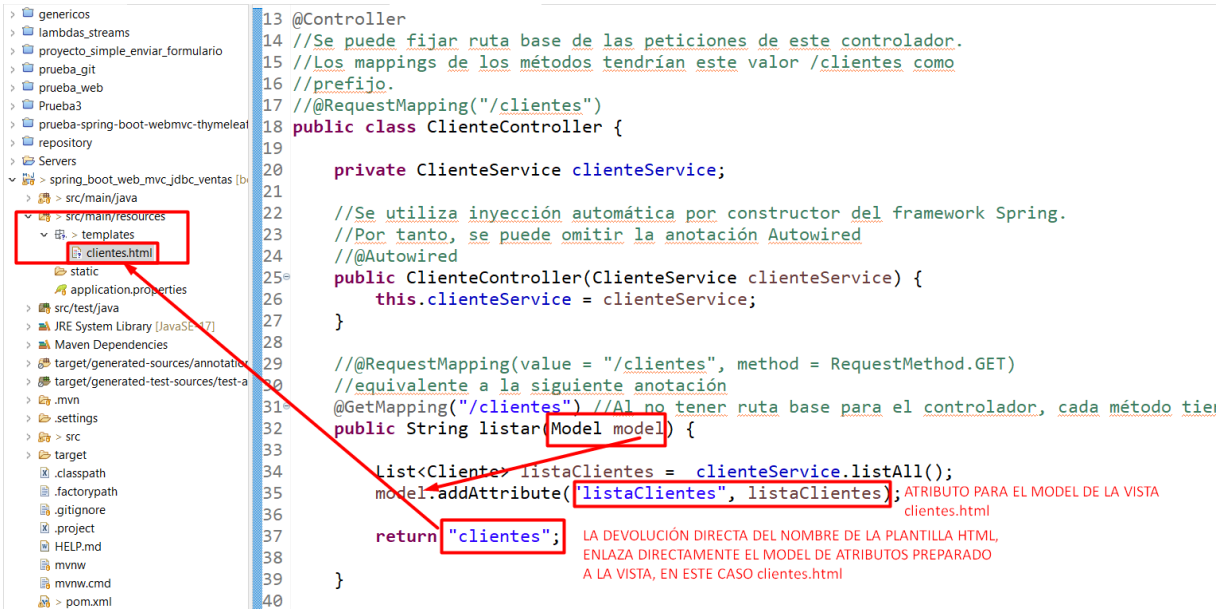
    /**
     * Inserta en base de datos el nuevo Cliente, actualizando el id en el bean Cliente.
     */
    @Override
    public synchronized void create(Cliente cliente) {

                            //Desde java15+ se tiene la triple quote """ para bloques de texto como cadenas.
        String sqlInsert = """
                            INSERT INTO cliente (nombre, apellido1, apellido2, ciudad, categoría)
                            VALUES  (     ?,         ?,         ?,       ?,         ?)
                           """;

        KeyHolder keyHolder = new GeneratedKeyHolder();
        //Con recuperación de id generado
        int rows = jdbcTemplate.update(connection -> {
            PreparedStatement ps = connection.prepareStatement(sqlInsert, new String[] { "id" });
            int idx = 1;
            ps.setString(idx++, cliente.getNombre());
```

Vista mediante plantillas thymeleaf

Cada método asignado a una ruta del Controlador tendrá que devolver, en última instancia, el nombre de la plantilla html con thymeleaf a la que se inyecta el modelo de atributos.




```java
@Controller
//Se puede fijar ruta base de las peticiones de este controlador.
//Los mappings de los métodos tendrían este valor /clientes como
//prefijo.
//@RequestMapping("/clientes")
public class ClienteController {

    private ClienteService clienteService;

    //Se utiliza inyección automática por constructor del framework Spring.
    //Por tanto, se puede omitir la anotación Autowired
    //@Autowired
    public ClienteController(ClienteService clienteService) {
        this.clienteService = clienteService;
    }

    //@RequestMapping(value = "/clientes", method = RequestMethod.GET)
    //equivalente a la siguiente anotación
    @GetMapping("/clientes") //Al no tener ruta base para el controlador, cada método tie
    public String listar(Model model) {

        List<Cliente> listaClientes =  clienteService.listAll();
        model.addAttribute("listaClientes", listaClientes);

        return "clientes";

    }
```

Si abrimos el fichero cliente.html podemos observar las etiquetas de thymeleaf sobre el html recogiendo los atributos del model:




```
<!DOCTYPE html SYSTEM "http://www.thymeleaf.org/dtd/xhtml1-strict-thymeleaf-4.dtd">
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<title>Clientes</title>
<style>
.clearfix::after {
    content: "";
    display: block;
    clear: both;
}

</style>
</head>
<body>
<body>

    <div id="contenedora" style="float:none; margin: 0 auto;width: 95%;" >
        <div class="clearfix">
            <div style="float: left; width: 50%">
                <h1>Clientes</h1>
            </div>
            <div style="float: none;width: auto;overflow: hidden;min-height: 80px;position: relative;">

                <div style="position: absolute; left: 39%; top : 39%;">

                        <form action="/clientes/crear">
                            <input type="submit" value="Crear">
                        </form>
                    </div>

            </div>
```

En la segunda mitad del fichero se pueden observar las etiquetas del motor de plantillas thymeleaf con su prefijo de espacio de nombres correspondiente **th:**

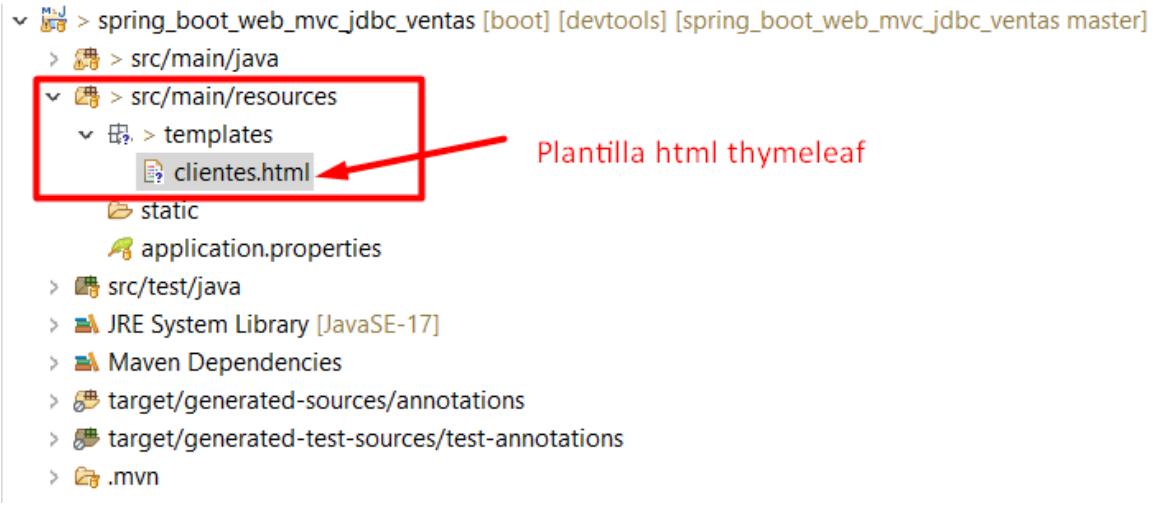



```
            <div style="float: left;width: 14%">Id</div>
            <div style="float: left;width: 14%">Nombre</div>
            <div style="float: left;width: 14%">Apellido1</div>
            <div style="float: left;width: 14%">Apellido2</div>
            <div style="float: left;width: 14%">Ciudad</div>
            <div style="float: left;width: 14%">Categoría</div>
            <div style="float: none;width: auto;overflow: hidden;">Acción</div>
        </div>
        <div class="clearfix">
            <hr/>
        </div>

        <div  th:each="cliente : ${listaClientes}" style="margin-top: 6px;" class="clearfix">
            <div style="float: left;width: 14%" th:text="${cliente.id}">0</div>
            <div style="float: left;width: 14%" th:text="${cliente.nombre}">nombre</div>
            <div style="float: left;width: 14%" th:text="${cliente.apellido1}">apellido1</div>
            <div style="float: left;width: 14%" th:text="${cliente.apellido2 ?: ' '}">apellido2</div>
            <div style="float: left;width: 14%" th:text="${cliente.ciudad}">ciudad</div>
            <div style="float: left;width: 14%" th:text="${cliente.categoria}">categoría</div>
            <div style="float: none;width: auto;overflow: hidden;">
                <form th:action="@{/clientes/__${cliente.id}__}" style="display: inline;">
                    <input type="submit" value="Ver Detalle" />
                </form>
                <form th:action="@{/clientes/editar/}+${cliente.id} " style="display: inline;">
                    <input type="submit" value="Editar" />
                </form>
                <form th:action="@{/clientes/borrar/__${cliente.}_id_}" method="post" style="display: inline;">
                    <input type="submit" value="Eliminar" />
                </form>
            </div>
        </div>
    </div>
```

Thymeleaf

Las plantillas html tendrás que crearlas dentro de la ruta **resources/templates/**



En el fichero html siempre tendrás que definir el espacio de nombres para thymeleaf, independiente del espacio de nombres de html:

clientes.html

```
1  <!DOCTYPE html SYSTEM "http://www.thymeleaf.org/dtd/xhtml1-strict-thymeleaf-4.dtd">
2  <html xmlns:th="http://www.thymeleaf.org">
3  <head>
4  <meta charset="UTF-8">
5  <title>Clientes</title>
6  <style>
7  .clearfix::after {
8      content: "";
9      display: block;
10     clear: both;
11 }
12
```

Realiza:

Tarea 3.1 - Creación de ComercialDAO con Spring JdbcTemplate, servicio, controlador y pruebas sobre CommandLineRunner de Spring

Entregar enlace a GitHub y archivo zip del proyecto, nombrado: ventas-apellidos.nombre.zip